

Today 11:24 AM

I'm participating in a 24 hour hackathon. Help me build a detailed, hour-by-hour project plan.

Project idea: "PacePilot" – Dynamic Athletic Performance & Recovery Orchestrator

Concept: An autonomous athletic coach that monitors streaming telemetry during workouts (pacing, cadence, heart rate zones, blood oxygen saturation) and dynamically modifies workout plans, nutrition intake, and recovery schedules on the fly.

- **Real-time Streaming Engine:** Micro-batch streaming via Databricks Lakeflow ingesting live workout telemetry. It calculates live metrics like Heart Rate Drift (cardiovascular drift) and fatigue indexes across sessions.
- **Agentic Multi-Step Workflow:**
 1. **Monitor & Analyze:** The agent continuously evaluates real-time effort against target zones.
 2. **Autonomous Decision:** If cardiovascular drift exceeds 10% (indicating dehydration or fatigue), the agent triggers a tool to dynamically recalculate the remaining workout target (e.g., lowering target wattage or pace).
 3. **API Integration:** The agent calls an external Nutrition API to generate immediate intra-workout refueling recommendations (e.g., *"Take 30g fast-acting carbs in 5 minutes based on your glycogen depletion rate"*).
 4. **Post-Session Re-scheduling:** After the session ends, the agent updates future training sessions in Google Calendar/TrainingPeaks via external APIs based on session strain scores.

Tech stack & APIs available:

- Build innovative software applications that leverage the Databricks platform and agentic AI to solve real-world problems for a wearable application with streaming data.

- Create an AI agent that analyzes data and provides actionable insights.
- Build an application that allows users to interact with and explore complex datasets using AI.
- Develop an agentic system that can use tools, data, and APIs to complete multi-step tasks.

Technologies:

Databricks Python SQL AI/MLLLMs agentic AI

Datasets & APIs: Wearable datasets containing multiple sessions will be provided.

My skills: [e.g., Python, React, SQL, ML]

Team size: [solo or N people]

Judging criteria: [if known — e.g., technical complexity, design, business value, presentation]

Please give me:

1. A **timeline** breaking down both days hour-by-hour, including buffer time, testing, and demo prep.
2. A **feature prioritization** split into Must-Have / Should-Have / Nice-to-Have, so I know what to cut if I fall behind.
3. A **technical architecture** recommendation — which API goes where, what the data flow looks like, and what to build first.
4. A **risk list** — the top 3-5 things most likely to go wrong, with a mitigation or fallback for each.
5. A **demo plan** — what the final 3-minute presentation should show, and which features matter most for the wow factor.

Be realistic about what's achievable in 2 days. Prioritize a working demo over a polished product.

Worked for 36s >

Build PacePilot around one complete story: a workout starts normally, effort drifts upward, the agent adjusts the remaining plan, finds a refueling option, and changes tomorrow’s training. That is achievable in 24 hours. A production wearable coach with validated fatigue prediction is a much larger project.

I’m assuming **one developer with Python, React, and SQL experience**, provided wearable datasets, and access to a Databricks workspace. The plan uses 24 elapsed hours across two days; shift the hours to match your kickoff.

Choose **one sport** after inspecting the dataset: cycling if it includes power, otherwise running if it includes speed. Replay the supplied sessions as a stream rather than connecting a physical wearable.

1. Hour-by-hour timeline

Each row covers one hour. Aim for a functioning end-to-end prototype by Hour 12.

Day 1 — establish the data pipeline and complete the core loop

Hour	Work	Concrete result
1	Check Databricks permissions, pipeline compute, SQL access, and available LLM endpoints. Inspect dataset columns and session lengths.	Confirmed environment and selected sport.
2	Pick one athlete and several sessions. Define units, identifiers, timestamps, workout targets, and the demo scenario. Attempt Google OAuth and obtain a nutrition API key.	Shared data contract; integration blockers identified early.
3	Build a Python replay script that releases timestamped JSON files in small batches. Add start, pause, reset, and replay-speed controls.	Recorded telemetry behaves like incoming data.

Hour	Work	Concrete result
4	Build Lakeflow ingestion into a Bronze Delta table using Auto Loader. Verify new files appear without duplicates.	Working Databricks streaming ingestion.
5	Create Silver data: normalize units, remove invalid records, deduplicate, and preserve athlete/session identifiers.	Clean telemetry available for calculations.
6	Calculate windowed HR, speed/power, zone time, baseline comparison, and data freshness. Check calculations on a hand-worked example.	Reliable metrics with an explicit drift proxy.
7	Buffer and meal break. Resolve pipeline latency or permissions. If Lakeflow is blocked, implement the Databricks notebook fallback.	Stable ingestion route and measured latency.
8	Build a minimal FastAPI backend exposing session state, metrics, current plan, and action history.	Frontend can retrieve actual processed data.
9	Build one React screen: telemetry chart, current target, drift indicator, and agent activity panel.	A workout can be watched live.
10	Implement bounded workout-adjustment tools and a trigger with persistence and cooldown. First test with deterministic inputs.	The remaining workout target changes exactly once.
11	Connect an LLM with tool calling. Give it metrics, workout state, constraints, and tools; execute validated calls and return tool results.	A real multi-step agent run with visible tool activity.
12	Integrate replay → Databricks → metrics → agent → updated target → UI. Run the whole scenario twice.	Core demo works end to end. Commit a stable checkpoint.

Day 2 — add external actions, harden, and prepare the presentation

Hour	Work	Concrete result
13	Connect food lookup to the agent. Use a configured fueling plan and API nutrient data to select a matching food or portion.	Refueling card with a source and explicit assumptions.
14	Compute a session effort summary and compare it with earlier sessions from the same athlete. Propose tomorrow's training adjustment.	Multi-session context influences recovery planning.
15	Connect Google Calendar and update one pre-created event in a dedicated demo calendar. If blocked, show a local before/after schedule.	Visible post-session scheduling action.
16	Integration buffer. Fix tool failures, inconsistent state, and nutrition serving-unit problems.	Full workflow succeeds from one reset.
17	Rest break. Save working state first.	Recovery time protected in the schedule.
18	Rest break.	No new scope.
19	Test normal effort, sustained drift, a brief HR spike, missing data, and repeated replay. Check duplicate external actions.	Evidence that the agent does more than fire on any high HR.
20	Test API timeout, LLM timeout, stale telemetry, and session end. Add clear failure states and fallback responses.	Failed calls cannot appear as successful actions.
21	Freeze features. Improve readability, chart labels, target before/after, and action status. Remove broken controls.	Presentation-ready screen.
22	Write the 3-minute script. Prepare the architecture explanation and submission README. Record a backup walkthrough.	Complete presentation and backup video.

Hour	Work	Concrete result
23	Rehearse twice with a timer. Reset between runs and confirm the demo calendar returns to its original state.	Repeatable demonstration under three minutes.
24	Final buffer and submission. Check links, screenshots, credentials, repository access, and required forms. Submit before the deadline.	Finished submission with a working fallback.

Cutoff rules

- **Hour 4:** If Lakeflow is blocked, ask a mentor immediately and switch temporarily to a Databricks Structured Streaming notebook. Clearly disclose the substitution.
- **Hour 12:** If the core loop is incomplete, stop external integration work until it works.
- **Hour 16:** Freeze integrations. Keep working APIs; replace blocked ones with clearly labeled local previews.
- **Hour 21:** Stop adding features.

With three people, divide ownership into **data pipeline**, **backend/agent/integrations**, and **frontend/demo**, while keeping these checkpoints.

2. Feature prioritization

Priority	Feature	Minimum acceptable version
Must-Have	Streaming telemetry in Databricks	Replay one supplied session into Delta tables in micro-batches.
Must-Have	Reliable live metrics	HR, speed or power, drift proxy, and freshness indicator.
Must-Have	Autonomous workout adjustment	A sustained trigger changes the remaining target within configured bounds.

Priority	Feature	Minimum acceptable version
Must-Have	Actual agent tool use	LLM requests tools; backend executes them and feeds results back.
Must-Have	Visible explanation and audit trail	Show observed metrics, tool arguments, results, and old/new targets.
Must-Have	Repeatable demo	Normal and drift scenarios, reset button, recorded backup.
Should-Have	External food lookup	One working API lookup, with cached data on failure.
Should-Have	Cross-session effort comparison	Compare today against a few earlier sessions for the same athlete.
Should-Have	Google Calendar update	Modify one event in a dedicated demo calendar.
Nice-to-Have	Dataset exploration through chat	Answer a few questions about past sessions using SQL.
Nice-to-Have	Cadence and SpO ₂ display	Show only when available; exclude from automatic decisions initially.
Nice-to-Have	Additional sports, voice, personalization	Add only after the full demo is stable.

Priority	Feature	Minimum acceptable version
Cut entirely for this hackathon	Custom ML training, physical wearable integration, TrainingPeaks onboarding, precise glycogen depletion prediction	Too much dependency or validation work for 24 hours.

TrainingPeaks access is restricted to approved developers, so use it only if the organizers already provide usable credentials.  TrainingPeaks H...

If judging requires an external API, promote **one** integration to Must-Have. Food lookup is the simpler default; Calendar creates the stronger visible action.

3. Recommended technical architecture

Use **Python + Lakeflow + Delta + FastAPI + React**, with one backend worker orchestrating the agent. Avoid Kafka, multiple agents, a vector database, and a custom trained model.

Lakeflow supports declarative streaming pipelines, and Auto Loader is an established ingestion route for incoming files. Confirm which execution mode and compute configuration the hackathon workspace supports before committing to a latency target.  Databricks on A... +1

Component	Responsibility	Implementation choice
Replay controller	Release recorded telemetry incrementally	Python writes uniquely named JSON batches to a supported storage location.
Bronze/Silver tables	Preserve raw data and clean it	Lakeflow pipeline; explicit schema and units.
Metrics table	Store window summaries and historical context	SQL/Python transformations with event timestamps.
Backend worker	Read new metrics and initiate agent runs	One Python process; remember the last processed window.
LLM endpoint	Choose appropriate tools and explain results	Accessible Databricks serving endpoint with function calling.
Workout tool	Change the remaining target	Backend validates limits and updates plan state.
Nutrition tool	Retrieve food composition	USDA FoodData Central or a provided API; cache a few foods.

Component	Responsibility	Implementation choice
Calendar tool	Change one future training event	Backend OAuth integration with Google Calendar.
React dashboard	Show measurements and completed actions	Poll your backend initially; add push updates only if needed.

Databricks supports function calling through compatible serving endpoints. Your application still executes the requested functions and returns their results to the model.  Databricks on A...

Define the metric before building the agent. For a simple demo, use:

$$\text{HR-rise proxy} = 100 \times \frac{\text{current window mean HR} - \text{baseline mean HR}}{\text{baseline mean HR}}$$

Compare a post-warm-up baseline with the latest window **only during approximately steady speed or power**. Otherwise, display “not comparable.” This is an explicitly defined prototype proxy, not a full cardiovascular-drift measurement.

Use the proposed **10% threshold as a configurable demo rule**, require it to persist across several updates, and suppress repeated adjustments with a cooldown. Drift alone does not establish dehydration or fatigue; research describes several contributing physiological factors.  PubMed

Likewise:

- Define session strain as a transparent **effort score**, such as minutes in each configured HR zone multiplied by documented weights.
- Compare sessions from the same athlete.
- Keep nutrition timing and carbohydrate targets in a configured fueling plan. A food API supplies nutrient composition; it does not infer glycogen depletion from heart rate. USDA’s API provides food search and nutrient information.  USDA FoodData...

The agent loop should be small and concrete:

1. A new eligible metrics window arrives.
2. The worker supplies current metrics, earlier sessions, existing plan, and constraints.
3. The agent calls `adjust_remaining_workout` .

- 4. The backend validates and applies the change, then returns the result.
- 5. The agent calls `lookup_refuel_option` using the configured fueling target.
- 6. At session end, it calls `propose_recovery_change` , then `update_demo_calendar` if enabled.
- 7. Every requested and completed action appears in the dashboard.

Use one action identifier per trigger so replaying or retrying cannot duplicate changes. Keep all credentials server-side. Restrict autonomous calendar writes to the dedicated demo calendar and its pre-created event; Google’s API supports event updates through authenticated requests.

 developers.goo... +1

Build first: one batch through Databricks, one trustworthy metric, one visible target change. That proves the central product before adding nutrition and scheduling.

4. Top risks and fallbacks

Risk	Mitigation	Fallback
Databricks access or streaming latency blocks progress	Test permissions and measure end-to-end latency immediately. Warm compute before presenting.	Databricks notebook processing; disclose any Lakeflow substitution. Recorded walkthrough if the workspace becomes unavailable.
Dataset lacks power, speed, SpO ₂ , or a useful drift episode	Inspect before choosing the sport. Use optional fields only when present.	Replay actual sessions plus a clearly labeled synthetic drift scenario. Without speed/power, call the metric HR rise rather than drift.
OAuth, API keys, or network access fail	Try authentication in Hour 2; set request timeouts and cache food data.	Local schedule preview and cached nutrition lookup, visibly labeled.
Agent produces invalid calls or repeats actions	Validate tool schemas, bound adjustments, cap tool-loop steps, and deduplicate by action ID.	Deterministic controller with a templated explanation; disclose that agent mode is unavailable.

Risk	Mitigation	Fallback
Scope creep prevents a complete demo	One athlete, one sport, one dashboard, one recovery event. Enforce the cutoff hours.	Drop chat, extra charts, SpO ₂ logic, and Calendar before sacrificing the working streaming-to-adjustment loop.

5. Three-minute demo plan

Time	Show	Explain
0:00–0:20	Workout target beside live telemetry	“Static plans stay fixed even when the athlete’s response changes. PacePilot adapts the plan during the session.”
0:20–0:45	Start accelerated replay; show recent data and normal effort	Identify the supplied dataset and label replay speed.
0:45–1:20	Sustained drift appears; show the agent’s tool call and target before/after	Explain the evidence and the configured adjustment rule.
1:20–1:45	Refueling card with retrieved food composition	Explain that it matches the athlete’s configured fueling plan.
1:45–2:15	End session; show historical effort comparison and tomorrow’s event change	Show recovery orchestration extending beyond the current workout.
2:15–2:40	Compact architecture and action log	Point out Databricks streaming, persistent session history, and actual tool execution.
2:40–3:00	Return to the completed workflow	State what works today and what needs validation before real athletic use.

The strongest wow moment is the visible chain of completed actions: telemetry changes → target changes → refueling option appears → tomorrow’s session changes.

Make each action display **why it happened, what changed, and whether it succeeded**. Keep a normal-effort replay ready to demonstrate that PacePilot also knows when to leave the plan alone.

we need a wearable component, it is part of the project outline. find some way to integrate it

Worked for 15s >

You’re right—I cut a required component. **Make live heart-rate data from a physical wearable a Must-Have**, and cut optional features to make room.

The most practical route: a Bluetooth heart-rate strap or armband connected directly to your laptop. A Polar H10 is one example that broadcasts HR over Bluetooth; borrow compatible hardware from teammates or organizers before buying anything.  Getting started

You don’t need every metric to come from the wearable. Live HR can power zone monitoring, while the supplied datasets provide historical session context. Add pace, cadence, or SpO₂ only if your actual device exposes them.

How to connect it

Use your React app’s **“Connect wearable”** button to request a Bluetooth device and subscribe to its standard Heart Rate Measurement notifications. The Bluetooth Heart Rate Service provides a standardized route for compatible sensors. Web Bluetooth requires a supported browser and secure context, so test on the actual demo laptop immediately.  [bluetooth.com](#) +1

Step	What you build
1. Connect	React requests the sensor, connects, and subscribes to HR notifications.
2. Capture	Decode BPM and attach receipt timestamp, athlete ID, session ID, and sequence number.
3. Upload	Buffer readings and POST small batches to your Python backend every few seconds.

Step	What you build
4. Ingest	Backend writes uniquely named JSON batches to storage monitored by Lakeflow Auto Loader.
5. Analyze	Databricks calculates HR-zone exposure and session metrics.
6. Act	Agent adjusts the workout recommendation; dashboard shows the completed action.

Keep Bluetooth on the user’s laptop. The remote Databricks environment receives uploaded readings; it does not connect to the sensor itself.

Show two separate statuses: **“Sensor connected”** and **“Latest Databricks analysis: X seconds ago.”** That makes ingestion delays visible.

Revise the first six hours

Hour	Revised task	Success condition
1	Inventory available wearables and test Bluetooth compatibility alongside Databricks access.	Identify a device you can actually read.
2	Build connection and HR decoding.	Real BPM appears on screen.
3	Add batching, backend upload, and session identifiers.	Wearable readings reach backend reliably.
4	Connect uploaded batches to Lakeflow.	Real wearable records appear in Delta.
5	Calculate zone exposure and add connection/freshness indicators.	Dashboard displays processed live metrics.
6	Connect one bounded workout-adjustment tool.	Wearable data can trigger a real recommendation change.

Move historical drift calculations later. **Cut dataset chat, elaborate nutrition personalization, and Calendar integration first if necessary.**

Keep the demo credible

Use two clearly labeled modes:

- **Live wearable:** Show a teammate's actual HR arriving through Databricks and influencing zone-based coaching.
- **Recorded-session replay:** Show sustained drift and the complete workout → nutrition → recovery workflow using a longer session.

A short stage demonstration won't establish cardiovascular drift. Without approximately steady workload data, label a live increase "**heart-rate rise**", and use zone-based coaching instead. Don't combine live HR with unrelated replayed pace and present it as a measured workout.

For the three-minute presentation, spend the first **30–45 seconds proving the physical sensor connection**, then switch explicitly to accelerated replay for the longer-term orchestration.

Which wearable models and phones/laptops does your team already have access to? That determines whether direct Bluetooth is available or we need a device-specific bridge.